Titanic Survival Prediction & Data Analytics Pipeline

An end-to-end Data Science and Machine Learning pipeline for the Titanic dataset built using **Python**, **scikit-learn**, **Pandas**, **NumPy**, and **Seaborn**.

This repository implements a production-grade machine learning workflow featuring automated data profiling, missing-value handling based on threshold rules, univariate and bivariate exploratory data analysis (EDA), multi-model classification, class-imbalance evaluation, hyperparameter tuning with Out-of-Bag (OOB) scoring, multivariate regression for fare prediction, and pipeline serialization (`joblib`).

Table of Contents

Key Features

- **Automated Data Profiling & Offline Fallback:** Dynamically loads data using Seaborn's dataset loader (`sns.load_dataset("titanic")`) or falls back to a local `titanic.csv` file.

- **Threshold-Based Missing Value Handling**:
  - Drops columns with excessive missingness (`deck` ~77.1%).
  - Imputes numerical features with median strategy (`age` ~19.8%).
  - Drops rows for minimal missingness (`embarked` / `embark_town` <0.5%).
- **Exploratory Analysis & Visualization**: Calculates IQR outlier bounds, central tendency metrics (mean, median, mode), skewness, and exports publication-ready charts.
- **Classification Modeling**:
  - Compares **Logistic Regression**, **Decision Tree** (with tree visualization export), and **Random Forest Classifiers**.
  - Evaluates performance using Accuracy, Precision, Recall, F1-Score, ROC-AUC, and Confusion Matrices.
- **Class Imbalance Evaluation**: Compares baseline performance against `class_weight="balanced"` and SMOTE oversampling (`imblearn` with fallback handling).
- **Hyperparameter Tuning**: Executes 5-fold `GridSearchCV` over a Scikit-Learn `Pipeline` combined with Random Forest Out-Of-Bag (OOB) scoring.
- **Fare Regression Side-Task**: Implements multivariate Linear Regression to predict fare prices and evaluates residuals for heteroscedasticity.
- **Pipeline Serialization & End-to-End Inference**: Persists the complete `ColumnTransformer` + `Classifier` state as `best_titanic_pipeline.joblib` and validates inference against raw, unpreprocessed input dictionary payloads.

Tech Stack

- **Language**: Python 3.10+
- **Data Processing**: `pandas` , `numpy`
- **Visualization**: `matplotlib` , `seaborn`
- **Machine Learning**: `scikit-learn`
- **Class Imbalance Handling**: `imbalanced-learn` (SMOTE)

- **Model Persistence:** `joblib`

Project Architecture & Task Breakdown

The script executes sequentially across 13 modular tasks:

Task ID	Function / Component	Description
Task 1	`load_and_profile_data()`	Dataset loading, shape check, summary statistics, and missing value profiling.
Task 2	`handle_missing_values()`	Applies strict threshold-based missing value cleaning strategies.
Task 3	`univariate_analysis()`	IQR outlier detection for Age & Fare, distribution histograms, boxplots, and central tendency analysis.
Task 4	`bivariate_analysis()`	Demographics survival breakdowns and restricted 6x6 Pearson correlation heatmap.
Task 5	`produce_data_story_charts()`	Generates 4 multivariate data story charts saved to the local working directory.
Task 6	`exploratory_standardization_check()`	Z-score sanity check $z = \frac{x-\mu}{\sigma}$ verifying zero mean ($\mu \approx 0$) and unit variance ($\sigma \approx 1$).
Tasks 7 & 8	`build_and_evaluate_classifiers()`	Stratified train-test split, `ColumnTransformer` pipeline, 3-model classifier comparison, decision tree diagram export, and ROC curve comparison.

Task 9	`evaluate_imbalance_handling()`	Evaluates baseline, `class_weight='balanced'`, and SMOTE oversampling on training folds.
Task 10	`tune_random_forest_with_oob()`	`GridSearchCV` hyperparameter tuning and Random Forest OOB score extraction.
Task 11	`fare_regression_side_task()`	Multivariate linear regression predicting fare price (MAE, $RMSE$, R^2, $AdjR^2$) and residual heteroscedasticity plot.
Tasks 12 & 13	`save_and_verify_pipeline()`	Exporting `best_titanic_pipeline.joblib` and testing raw end-to-end inference reload.

Generated Output Artifacts

Running the script automatically generates the following visual and model artifacts in the script directory:

```
.
├── titanic.csv                          # Local offline CSV backup
├── best_titanic_pipeline.joblib         # Exported end-to-end trained model pipeline
├── univariate_age_fare.png              # Age & Fare distribution histograms + boxplots
├── correlation_heatmap.png              # 6x6 Feature correlation matrix heatmap
├── chart1_survival_class_sex.png        # Survival rate by passenger class and gender
├── chart2_fare_age_survival.png         # Fare vs Age scatter plot segmented by survival & class
├── chart3_family_size_survival.png      # Survival rate by family size (SibSp + Parch + 1)
├── chart4_embarked_class_survival.png   # Survival rate by port of embarkation and class
├── decision_tree_visualization.png      # Rendered Decision Tree diagram (Max Depth = 4)
├── roc_curves_comparison.png            # ROC Curves comparison across tested models
└── fare_regression_residuals.png        # Residual plot highlighting regression heteroscedasticity
```

Installation & Setup

1. **Clone or download this repository:**

```
git clone https://github.com/your-username/titanic-analytics-pipeline.git
cd titanic-analytics-pipeline
```

2. **Create and activate a virtual environment (optional but recommended):**

```
python -m venv venv
# Linux/macOS:
source venv/bin/activate
# Windows:
venv\Scripts\activate
```

3. **Install required dependencies:**

```
pip install pandas numpy matplotlib seaborn scikit-learn joblib imbalanced-learn
```

Usage

Execute the entire end-to-end analytics and machine learning pipeline with a single command:

```
python main.py
```

Inference with Saved Pipeline

You can load `best_titanic_pipeline.joblib` directly to make predictions on raw, un-preprocessed passenger records:

```
import joblib
import pandas as pd

# 1. Load the fitted pipeline artifact
model = joblib.load("best_titanic_pipeline.joblib")

# 2. Define raw passenger data (no manual scaling or encoding required)
raw_passenger = pd.DataFrame([{
    "pclass": 1,
    "sex": "female",
    "age": 29.0,
    "sibsp": 0,
    "parch": 0,
    "fare": 211.33,
    "embarked": "S"
}])

# 3. Predict class and probabilities
prediction = model.predict(raw_passenger)[0]
probabilities = model.predict_proba(raw_passenger)[0]

print(f"Prediction: {'Survived' if prediction == 1 else 'Died'}")
print(f"Probability (Died): {probabilities[0]:.4f}")
print(f"Probability (Survived): {probabilities[1]:.4f}")
```

Model Evaluation Summary

- **Stratified Splitting**: 80/20 train-test split maintains identical target survival distribution (~38.38%) across sets.

- **Column Transformer Pipeline**: Applied `SimpleImputer(strategy="median")` + `StandardScaler()` for continuous features and `SimpleImputer(strategy="most_frequent")` + `OneHotEncoder()` for categorical features.
- **Tuned Random Forest Estimator**: Hyperparameter tuning via 5-fold cross-validation selects optimal tree depth, feature subsampling, and tree count, backed by Out-of-Bag (OOB) validation scoring.

License